

EXECUTIVE SUMMARY

Target: http://multi-vuln.com
Scan ID: TORTURE-P2-7bd0e61f
Scan Date: 2026-03-04 17:31:53
Scan Duration: N/A
Total Requests Sent: 29
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 5 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 4 | Medium: 1 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- High severity vulnerabilities present significant security risks across all assessed systems;
- Immediate action required to patch or mitigate identified high-risk issues;
- Focus remediation efforts on resolving 4 out of 5 total vulnerabilities first;
- Long-term strategy should include regular vulnerability monitoring and mitigation planning.

EXECUTIVE STRATEGIC IMPACT

```
{
  "strategic_attack_path_analysis": [
    "An attacker could exploit the CSRF vulnerability to perform unauthorized actions on behalf of legitimate users, potentially gaining elevated privileges and access to sensitive data.",
    "By chaining IDOR vulnerabilities across multiple systems or services, an adversary can manipulate resource permissions to extract confidential information, modify critical configurations, or even escalate their own privileges within the compromised environment.",
    "The PROMPT_INJECTION flaw could be leveraged to inject malicious commands into user interactions with web applications, enabling the attacker to execute arbitrary code on vulnerable endpoints and further compromise system integrity. By combining these techniques in a coordinated manner, an adversary can achieve a high-impact objective such as full system compromise."
  ]
}
```

DETAILED FINDINGS

FILTER: [ANSWER] AUTHENTICATION GATES

Finding #1: Cross-Site Request Forgery

HIGH

CWE: CWE-352
CVSS Score: 7.5 (HIGH)

THREAT SCORE: 75/100

Description:

- The target URL `http://test-target-torture-.com/api/v1/endpoint_0` is susceptible to Cross-Site Request Forgery (CSRF) attacks.
- An attacker could trick a user into performing unintended actions on the vulnerable endpoint by embedding malicious requests in web pages or emails.
- User authentication and session management are insufficiently validated, allowing forged requests to bypass security controls.

Impact:

- Sensitive data may be exposed or modified through unauthorized actions performed by attackers.
- Business processes could be disrupted due to unintended transactions initiated by CSRF attacks.
- Reputation damage for the affected system as user trust is compromised and potential financial losses incurred from fraudulent activities.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_0
URL: `http://test-target-torture-.com/api/v1/endpoint_0`
Analysis: The application failed to validate and sanitize user input, allowing malicious

requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Cross-Site Request Forgery
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement proper anti-CSRF tokens in all request handling endpoints, ensuring each request includes a unique token that must be verified on receipt.
- Validate and sanitize input data thoroughly to prevent malicious payloads from being injected into requests.
- Enforce strict authentication mechanisms such as multi-factor authentication for critical operations involving the vulnerable endpoint.

Recommended Code Fix:

```
// Example of adding anti-CSRF protection in Node.js using express  
const csrf = require('csrf');
```

```
app.use(csrf({ cookie: true }));  
// Ensure each request includes a valid CSRF token before processing
```

FILTER: OBJECT REFERENCES (IDOR)

Finding #2: Insecure Direct Object Reference (IDOR)

CRITICAL

CWE: CWE-639
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- The endpoint allows direct access to sensitive data via the 'id' parameter.
- User input is not properly validated or sanitized before being used in queries.
- No authentication or authorization checks are enforced for this resource.

Impact:

- Unauthorized users can retrieve confidential information.
- Data integrity may be compromised by unauthorized modifications.
- Potential exposure of sensitive user data to malicious actors.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'IDOR' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The application failed to properly validate and sanitize user input, allowing arbitrary data to be passed in the URL.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Insecure Direct Object Reference (IDOR)

Raw Payload: test_payload_1

Encoded: 746573745f7061796c6f61645f31

Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement input validation and sanitization on the 'id' parameter.
- Add authentication checks (e.g., API keys, JWT tokens) before accessing this endpoint.
- Restrict access to authorized users only through proper authorization mechanisms.

Recommended Code Fix:

```
// Example secure code snippet in Python
if request.method == 'GET':
    id_value = validate_id(request.args.get('id'))
    response_data = get_sensitive_data(id_value)
else:
    return forbidden()
```

FILTER: INJECTION & FUZZING

Finding #3: AI Prompt Injection

CRITICAL

CWE: CWE-77
CVSS Score: 9.0 (CRITICAL)

THREAT SCORE: 90/100

Description:

- The endpoint is vulnerable to prompt injection attacks, allowing malicious input to manipulate the response.
- Attackers can inject arbitrary commands or data into API responses by manipulating user-provided parameters.
- This vulnerability could lead to unauthorized actions, data leakage, and potential system compromise.

Impact:

- Unauthorized access to sensitive resources
- Data exfiltration through manipulated responses
- Potential escalation of privileges

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PROMPT_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_2
URL: http://test-target-torture-.com/api/v1/endpoint_2
Analysis: The application failed to properly validate and sanitize user input, allowing malicious payloads to be injected into API requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.

Result	200 OK	Server returns data for unauthorized request.
--------	--------	---

Payload Specifications:

Vector Category:	AI Prompt Injection
Raw Payload:	test_payload_2
Encoded:	746573745f7061796c6661645f32
Encoding Type:	None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json
{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement input validation and sanitization for all user-provided parameters.
- Use parameterized queries or secure APIs that prevent string concatenation with untrusted data.
- Enforce strict access controls and authentication mechanisms to restrict unauthorized endpoints.

Recommended Code Fix:

```
// Example of a code fix using parameterized query (Java example)
// Assuming an existing JDBC connection
String sql = "SELECT * FROM users WHERE id=?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, userId);
pstmt.executeQuery();
```


Finding #4: AI Prompt Injection

CRITICAL

CWE: CWE-77
CVSS Score: 9.0 (CRITICAL)

THREAT SCORE: 90/100

Description:

- The endpoint /api/v1/endpoint_4 is susceptible to prompt injection attacks.
- Attackers can manipulate input parameters to inject malicious prompts into the system.
- This vulnerability could lead to unauthorized data access, command execution, or privilege escalation.

Impact:

- Potential compromise of sensitive user data stored by the application.
- Execution of arbitrary commands on the server due to prompt injection.
- Unauthorized actions performed with elevated privileges leading to potential system breach.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PROMPT_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_4
URL: http://test-target-torture-.com/api/v1/endpoint_4
Analysis: The application failed to properly validate and sanitize user input, allowing malicious payloads to be injected into API requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_4	test_payload_4	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: AI Prompt Injection

Raw Payload: test_payload_4

Encoded: 746573745f7061796c6f61645f34

Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_4'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_4 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement input validation and sanitization for all parameters sent to /api/v1/endpoint_4.
- - Use parameterized queries or prepared statements to prevent direct execution of user-supplied data.
- - Employ a robust web application firewall (WAF) with prompt injection detection rules.

Recommended Code Fix:

```
// Example secure code snippet using parameterized query in Java
```

FILTER: SSRF

Finding #5: Server-Side Request Forgery

CRITICAL

CWE: CWE-918
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- Server-Side Request Forgery vulnerability in endpoint /api/v1/endpoint_3.
- Attackers can manipulate the request URL to make internal services or external servers vulnerable.
- Potential for unauthorized data access, service disruption, and information leakage.

Impact:

- Unauthorized data exposure from compromised endpoints.
- Service interruption due to misdirected requests.
- Exposure of sensitive information through leaked responses.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_3
URL: http://test-target-torture-.com/api/v1/endpoint_3
Analysis: The application failed to validate and sanitize user input, allowing arbitrary URLs to be injected into API requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_3	test_payload_3	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Server-Side Request Forgery

Raw Payload: test_payload_3

Encoded: 746573745f7061796c6f61645f33

Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement strict input validation and sanitization for all external URLs.
- - Restrict outbound network access to only necessary services.
- - Deploy rate limiting and anomaly detection mechanisms on the vulnerable endpoint.

Recommended Code Fix:

```
// Example of a secure code snippet using curl with URL verification
const fetch = require('node-fetch');

async function makeRequest(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  } catch (error) {
    console.error('SSRF detected:', error.message);
    // Handle or block the request
  }
}

// Usage example
makeRequest('http://test-target-torture-.com/api/v1/endpoint_3');
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 17:31:53

[Sigma] JOB_ASSIGNED - 2026-03-04 17:31:53

[Alpha] LOG - 2026-03-04 17:31:54

[Alpha] LOG - 2026-03-04 17:31:54

[Kappa] LOG - 2026-03-04 17:31:54

[Kappa] LOG - 2026-03-04 17:31:54

[Beta] LOG - 2026-03-04 17:31:54

[Alpha] LOG - 2026-03-04 17:31:54

[Gamma] LOG - 2026-03-04 17:31:54

[Beta] LOG - 2026-03-04 17:31:54

[Beta] LOG - 2026-03-04 17:31:54

[Beta] LOG - 2026-03-04 17:31:54

[Alpha] LOG - 2026-03-04 17:31:55

[Gamma] LOG - 2026-03-04 17:31:55

[Gamma] LOG - 2026-03-04 17:31:55

[Beta] LOG - 2026-03-04 17:31:55

[Kappa] LOG - 2026-03-04 17:31:55

[Gamma] LOG - 2026-03-04 17:31:55

[Beta] LOG - 2026-03-04 17:31:55

[Beta] LOG - 2026-03-04 17:31:55

[Kappa] LOG - 2026-03-04 17:31:55

[Beta] LOG - 2026-03-04 17:31:55

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:55

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:56

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:56

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:57

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:57

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:57

[Kappa] GI5_LOG - 2026-03-04 17:32:03